

PRÁCTICA 2

Hecho por: Enrique Gómez Fernández y Daniel Birsan, P10

I-C. Cuestiones sobre CDs y CCs

1. Sin darnos cuenta, en nuestro algoritmo de encontrar CCs podemos pasar listas con ramas repetidas o donde los vértices coinciden con los de una que ya está aunque en orden inverso. ¿Afectará esto al resultado del algoritmo? ¿Por qué?

No, el resultado es el mismo porque las aristas no afectan en la compresión de caminos, además, la mayoría se intercambian para completar la compresión. Por otro lado a los vértices repetidos o no se les aplica find, por tanto, estén repetidos o no, pasarán a estar unidos a cierto representante.

2. Argumentar que nuestro algoritmo de encontrar componentes conexas es correcto, esto es, que a su final en los distintos subconjuntos disjuntos se encuentran los vértices de las distintas componentes del grafo dado.

Observando el código fuente tenemos que a cada vértice se le aplica un find y posteriormente se realiza la unión de los resultados anteriores. Por ello podemos deducir que el grafo se organizará de tal forma que todos los CDs tendrán como representante al representante del grafo, por tanto se ve claramente que al aplicar ccs utilizando la compresión de caminos todos los CDs serán los vértices del grafo.

3. El tamaño de un grafo no dirigido viene determinado por el número de nodos y la longitud de la lista l de ramas. Estimar razonadamente en función de ambos el coste del algoritmo de encontrar las componentes conexas mediante conjuntos disjuntos.

Si en un CD con N elementos hacemos L uniones por rango y $M = \Omega(N)$ finds con compresión de caminos, el coste general es $O(L + M \lg^* N)$. Esto lo deducimos a raíz de ver el código fuente en el que ejecutamos L veces find y union.

II-B. Cuestiones sobre la solución greedy de TSP.

1. Estimar razonadamente en función del número de nodos del grafo el coste codicioso de resolver el TSP. ¿Cuál sería el coste de aplicar la función *exhaustive_tsp* ? ¿Y el de aplicar la función *repeated_greedy_tsp* ?

El coste será $N!$ para la *exhaustive*, en cambio, para la función *repeated* el coste será N^3 . Esto lo podemos comprobar midiendo los tiempos que tarda en ejecutarse cada algoritmo.

Para el *exhaustive*, hemos medido con un cronómetro cuánto tardaba para los valores 8, 9, 10 y 11: N t(s)

8	0,2
9	1.2
10	12
11	128

Como vemos, los valores se ajustan a $N!$ ya que cada vez que se incrementa el valor, el tiempo de ejecución se multiplica por ese número de manera aproximada.

Para el *repeated_greedy*, hemos medido con un cronómetro cuánto tardaba para los valores 200, 400 y 500: N t(s)

200	0.8
400	6
500	11

Como vemos esto se ajusta a N^3 ya que si hacemos la relación, $400^3/200^3=6/0.8$ al igual que pasa con $500^3/200^3=11/0.8$ y por supuesto también con $500^3/400^3=11/6$.

2. A partir del código desarrollado en la práctica, encontrar algún ejemplo de grafo para el que la solución greedy del problema TSP no sea óptima.

Probando con la práctica, hemos llegado a que con la matriz

0	26	47	30
26	0	38	22
47	38	0	19
30	22	19	0

El resultado greedy no es el más óptimo, pues probando con greedy nos sale el circuito [0, 1, 3, 2, 0] con una distancia de 114 mientras que con el *exhaustive*, nos daba el circuito [0, 1, 2, 3, 0] con una distancia de 113. Por tanto en ese caso la solución greedy no es óptima, ya que no es la mejor.